



Technische Universität Berlin
Fakultät IV - Elektrotechnik und Informatik
Institut für Energie- und Automatisierungstechnik
Fachgebiet Elektrische Energiespeichertechnik

Projektdokumentation / Praktikumsdokumentation
State Estimation and Diagnosis of Battery Systems
WiSe 25/26

Group Work I: Model-Based SOC Estimation

vorgelegt von:

[REDACTED] ([REDACTED])

[REDACTED] ([REDACTED])

Removed personal data

[REDACTED] ([REDACTED])

Franz Wendel ([REDACTED])

Verantwortliche Hochschullehrerin:
Betreuer:
Tag der Einreichung:

Prof. Dr.-Ing. Julia Kowal
Jiaqi Yao
June 29, 2026

Contents

1. Task 1	1
2. Task 2	3
3. Task 3	10
Bibliography	18
Appendix A. Matlab-Code	20
Appendix B. Simulink Models	24

1. Task 1

a) Battery Pack Configuration

The LG cell used in this design has a nominal voltage of 3.6 V and a nominal capacity of 2.5 Ah [1]. The required battery pack specifications are 36 V nominal voltage and 10 Ah nominal capacity.

Series connection

The number of cells connected in series is determined by

$$n_s = \frac{U_{\text{pack}}}{U_{\text{cell}}} = \frac{36 \text{ V}}{3.6 \text{ V}} = 10. \quad (1.1)$$

Parallel connection

The number of cells in parallel is

$$n_p = \frac{C_{\text{pack}}}{C_{\text{cell}}} = \frac{10 \text{ Ah}}{2.5 \text{ Ah}} = 4. \quad (1.2)$$

Thus, the battery pack configuration is **10s4p**, consisting of a total of 40 cells.

Nominal energy

The pack's nominal energy is calculated as

$$E_{\text{pack}} = U_{\text{pack}} \cdot C_{\text{pack}} = 36 \text{ V} \cdot 10 \text{ Ah} = 360 \text{ Wh}. \quad (1.3)$$

b) Interconnection and Measurement Concept

To minimize the number of measurement devices in the battery management system (BMS), the cells are arranged as parallel blocks connected in series. Each of the 10 series stages consists of 4 cells in parallel, forming 10 identical *4P blocks*. In a parallel

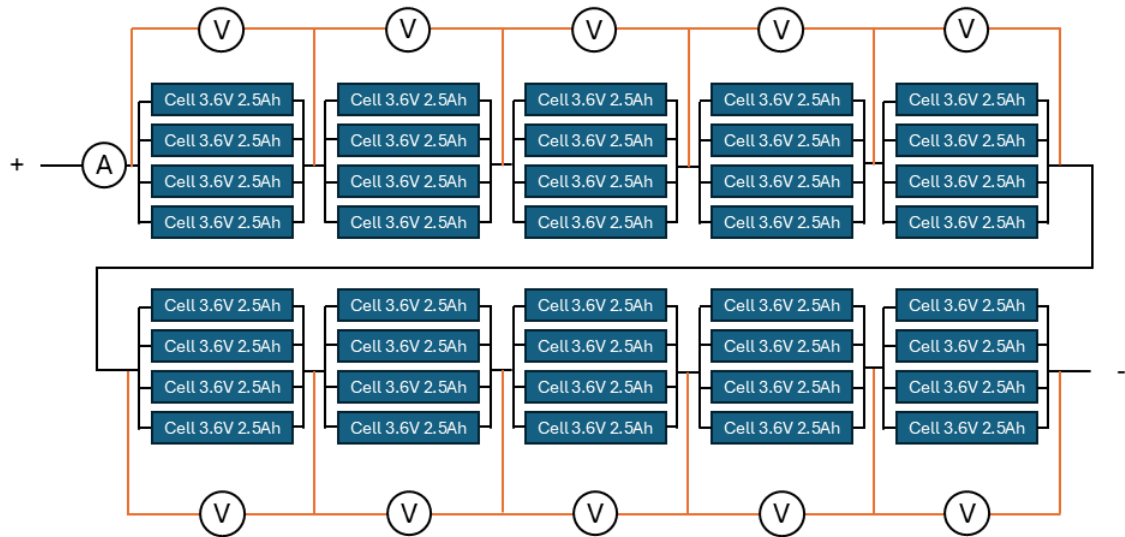


Figure 1.1.: Battery interconnection and measurement layout.

block, all cells share the same voltage, allowing a single voltage measurement per block.

A single current sensor is placed in series with the complete pack to measure the total pack current. Since all series-connected parallel blocks carry the same current and the individual cell currents in each parallel block can be assumed to be equally distributed, as given by the task, a single current sensor is sufficient to get a current measurement for all cells.

2. Task 2

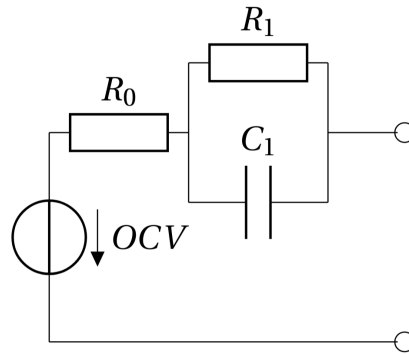


Figure 2.1.: Equivalent Circuit Model of the Cell

a)

To determine the differential equation for the given equivalent circuit model, Kirchhoff's voltage law is first applied to find all voltages in the circuit.

$$U_T = U_{OC} + U_0 + U_1 \quad (2.1)$$

Using Ohm's law U_0 can be written with the input current and the resistance R_0

$$U_T = U_{OC} + R_0 \cdot I + U_1 \quad (2.2)$$

With Kirchhoff's current law and the current equation for capacitors, the voltage drop over the RC-element can be substituted

$$U_T = U_{OC} + R_0 \cdot I + R_1 \cdot \left(I - C_1 \cdot \frac{dU_1}{dt} \right) \quad (2.3)$$

b)

In what follows, the previously derived equations are transformed into the following form:

$$\dot{x} = Ax + Bu \quad (2.4)$$

$$y = Cx + Du \quad (2.5)$$

$$\begin{bmatrix} \dot{SOC} \\ \dot{U}_1 \end{bmatrix} = A \begin{bmatrix} SOC \\ U_1 \end{bmatrix} + BI \quad (2.6)$$

$$U_T = C \begin{bmatrix} SOC \\ U_1 \end{bmatrix} + DI \quad (2.7)$$

The coulombic efficiency for this task is given as:

$$\eta = \begin{cases} 0.985 & \text{if } I \geq 0 \\ 1 & \text{if } I < 0 \end{cases} \quad (2.8)$$

The SOC is derived by the coulomb counting method

$$SOC = SOC(t=0) + \frac{\eta}{C_N} \int I(t) dt \quad (2.9)$$

Thus, the derivation of the SOC is

$$\dot{SOC} = \frac{d}{dt} \left(\frac{\eta}{C_N} \int I(t) dt \right) = \frac{\eta}{C_N} I(t) \quad (2.10)$$

U_1 was previously defined as

$$U_1 = R_1 \cdot \left(I - C_1 \cdot \frac{dU_1}{dt} \right) \quad (2.11)$$

To get the derivative of U_1 the equation can be rearranged for $\frac{dU_1}{dt}$

$$\dot{U}_1 = \frac{dU_1}{dt} = -\frac{1}{R_1 C_1} U_1 + \frac{1}{C_1} I \quad (2.12)$$

With these values, matrices A and B can be calculated

$$\begin{bmatrix} \dot{SOC} \\ \dot{U}_1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & -\frac{1}{R_1 C_1} \end{bmatrix} \begin{bmatrix} SOC \\ U_1 \end{bmatrix} + \begin{bmatrix} \frac{\eta}{C_N} \\ \frac{1}{C_1} \end{bmatrix} I \quad (2.13)$$

With the definition, that the open-circuit voltage is a function that depends on SOC

$U_{OC} = f(SOC)$ the previous formula of U_T can be rewritten with matrices C and D .

$$U_T = U_{OC} + R_0 \cdot I + U_1 = \begin{bmatrix} \frac{f(SOC)}{SOC} & 1 \end{bmatrix} \begin{bmatrix} SOC \\ U_1 \end{bmatrix} + \begin{bmatrix} R_0 \end{bmatrix} I \quad (2.14)$$

c)

Let $R_{0,cell}$, $R_{1,cell}$, $\tau_{1,cell}$ and $C_{N,cell}$ be the parameters for a single cell, as provided in `CellParameter.mat`. The equivalent parallel pack parameters are as follows:

- Ohmic resistance of n_p identical cells in parallel:

$$R_{0,parallel} = \frac{R_{0,cell}}{n_p} \quad (2.15)$$

- RC resistance in parallel:

$$R_{1,parallel} = \frac{R_{1,cell}}{n_p} \quad (2.16)$$

- Time constant of each RC branch does not change in a parallel configuration:

$$\tau_{1,parallel} = \tau_{1,cell} \quad (2.17)$$

- Nominal capacity: sum of cell capacities in parallel:

$$C_{N,parallel} = n_p C_{N,cell} \quad (2.18)$$

With the given nominal capacity $C_{N,cell}$ from the technical data sheet [1], the parallel equivalent capacity $C_{N,parallel}$ can be calculated and converted to Ampere seconds.

$$C_{N,cell} = 2.5 \text{ A h} \cdot 3600 \frac{\text{s}}{\text{h}} = 9000 \text{ A s} \quad (2.19)$$

$$C_{N,parallel} = 4 \cdot 9000 \text{ A s} = 36000 \text{ A s} \quad (2.20)$$

The script `InitParameterParallModel.m` (A.1) implements these relations using the data from `CellParameter.mat`. It sets up a structure `par_model` that is later used by the Simulink model. This script is called from the model's `InitFcn` callback so that `par_model` is available in the base workspace before the simulation starts.

d)

Using `InitParameterParallModel.m` (A.1), the SOC-dependent parameters are loaded from `CellParameter.mat` and stored in the struct `par_model`. The parallel-equivalent model is implemented in Simulink in a state-space form using vector and matrix operations. Because C_1 is not given we need to reformulate the matrices to use τ_1 instead. η is selected depending on the current direction.

$$A = \begin{bmatrix} 0 & 0 \\ 0 & -\frac{1}{\tau} \end{bmatrix}, \quad B = \begin{bmatrix} \frac{\eta}{C_N} \\ \frac{R_1}{\tau_1} \end{bmatrix}, \quad (2.21)$$

$$C(\text{SOC}) = \begin{bmatrix} \frac{f(\text{SOC})}{\text{SOC}} & 1 \end{bmatrix}, \quad D = R_0. \quad (2.22)$$

Key elements of the implementation:

- **Initialization:** `InitParameterParallModel.m` is executed via the model *InitFcn* callback to load the parameter data and to compute the parallel-equivalent values (`par_model.R0`, `par_model.R1`, `par_model.Tau1`, `par_model.CN`, `par_model.eta`).
- **Current input:** a subsystem (“Current Cycle”) generates the input current $I(t)$ based on the *SOC* of the battery model. To implement this behavior a series of switch and comparator blocks are used in combination with a *Unit Delay* as shown in the Appendix 2.2. According to Task 2e the battery is first charged with 12.5 A until a *SOC* of a 100% is reached and then discharged with 24 A until its *SOC* reaches 1%.
- **SOC-dependent parameter update:** The signals $R_0(\text{SOC})$, $R_1(\text{SOC})$, $\tau_1(\text{SOC})$, $U_{OC}(\text{SOC})$ are computed using 1-D lookup tables driven by the current SOC estimate. This yields time-varying matrices $A(\text{SOC})$ and $B(\text{SOC})$ that are updated online during simulation.
- **State derivative computation:** In each time step the matrices $A(\text{SOC})$ and $B(\text{SOC})$ are assembled from the lookup outputs. The derivative is then computed using equation (2.13). In Simulink this is implemented using *Matrix Multiply* and vector sum blocks.
- **Integration:** A vector *Integrator* block integrates $\dot{x}$ to obtain $x(t)$. The integrator initial condition is set to `par_model.x_init`. The resulting SOC component is fed back to the lookup tables to update the SOC-dependent parameters.

- **Output equation:** The terminal voltage is computed using the algebraic output equation (2.14), where U_1 is extracted from the state vector. This is implemented using a product block for $R_0 \cdot I$ and a sum block to combine U_{OC} , U_1 and the ohmic contribution.
- **Logging:** Scopes and to workspace blocks are used to record relevant parameters for post-processing.

Due to the SOC dependency, $U_{OC}(SOC)$, $R_0(SOC)$, $R_1(SOC)$ and $\tau_1(SOC)$ are computed from a lookup table instead of being used explicitly within a matrix in Simulink. An image of the Simulink model can be found in the Appendix 2.1.

e)

Figures 2.2 and 2.3 show the simulated state of charge, current profile, and the corresponding voltage signals during a constant-current charge and discharge cycle. The simulation starts at a low state of charge of $SOC = 1\%$. During the charging phase, a constant current of $I = 12.5\text{ A}$ is applied. As expected from coulomb counting, the SOC increases linearly with time, confirming the correct implementation of the SOC state equation. The open-circuit voltage U_{OC} rises non-linearly with SOC , following the characteristic OCV–SOC relationship of the cell. This non-linear increase and the resulting shape of the voltage curves reflect the electrochemical behavior of the battery. The terminal voltage U_t exceeds the open-circuit voltage during charging due to charging overvoltages caused by the ohmic resistance R_0 and the RC network (R_1, C_1). The instantaneous ohmic overvoltage $U_0 = R_0 \cdot I$ appears immediately when the current is applied, while the RC overvoltage U_1 builds up more slowly as the capacitor C_1 charges. This results in a gradual increase of the total overvoltage during the constant-current charging phase.

When the operating mode is switched from charging to discharging, the current changes sign abruptly. This leads to an immediate drop in the terminal voltage due to the reversal of the ohmic overvoltage, followed by an exponential relaxation of the RC overvoltage governed by the time constant τ_1 . The transient behavior observed in U_t and U_1 is characteristic of first-order RC dynamics and clearly illustrates the separation between instantaneous and dynamic overvoltage effects.

During the discharge phase ($I < 0\text{ A}$), the SOC decreases linearly back to 1%, again confirming consistent coulomb counting. In this regime, the terminal voltage U_t lies below the open-circuit voltage, as the internal resistive and dynamic overvoltages now act in the opposite direction compared to charging. The voltage evolution follows the OCV–SOC curve, modulated by SOC-dependent internal parameters.

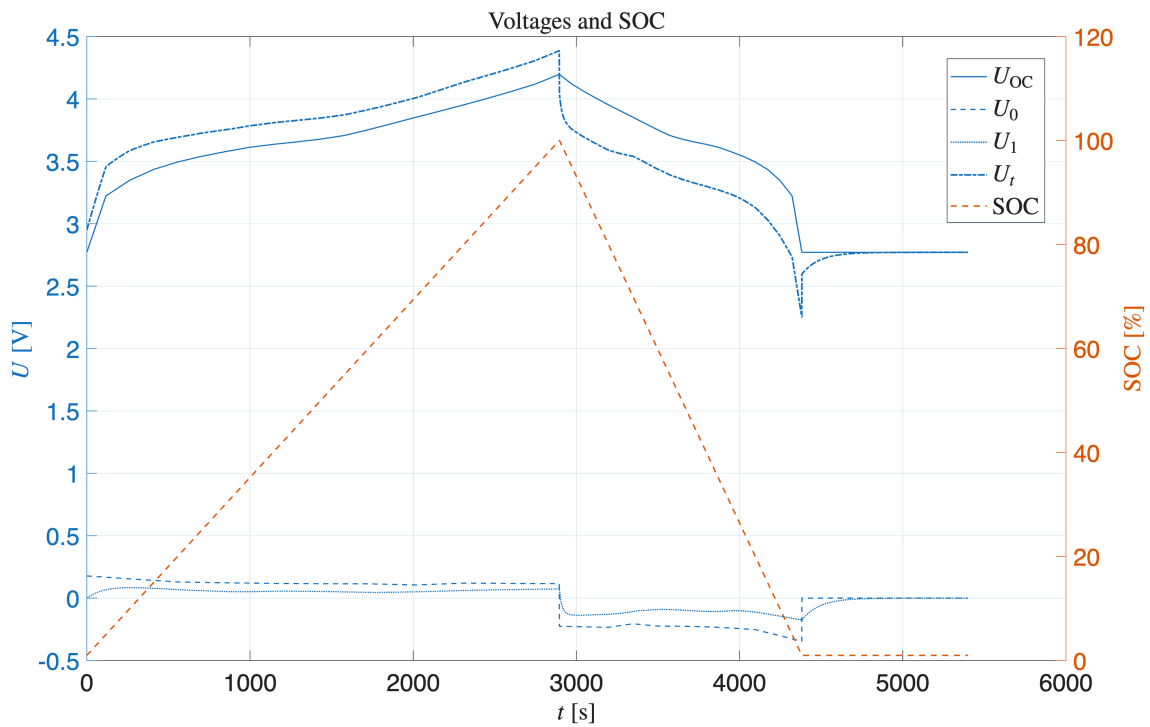


Figure 2.2.: Voltages and SOC over time

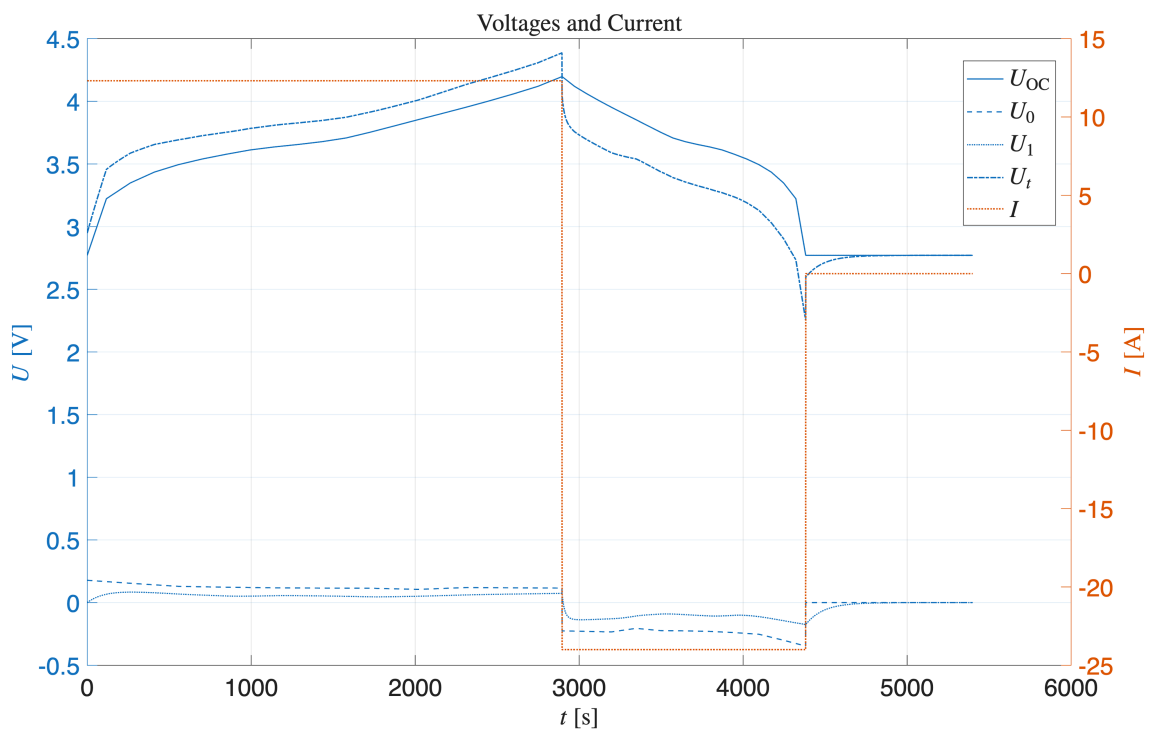


Figure 2.3.: Voltages and current over time

Small non-smooth transitions in the voltage signals are observed over the *SOC* range. These originate from the SOC-dependent parameterization of R_0 , R_1 and τ_1 using lookup tables. As the *SOC* crosses discrete support points, the internal parameters change in a piecewise manner, which directly affects both the state dynamics and the resulting overvoltages. This behavior reflects the data-driven nature of the model rather than numerical artifacts. Overall, the simulation exhibits physically consistent behavior: *SOC* remains within its bounds, voltage dynamics are stable, and the observed overvoltage effects during charging, discharging, and current reversal are in agreement with the equivalent circuit battery model.

3. Task 3

a)

To include a Kalman filter into the model it is necessary to change the form from a continuous state-space model to a discrete one. The following changes were made:

$$A \rightarrow A_d = \begin{bmatrix} 1 & 0 \\ 0 & e^{-\frac{\Delta t}{\tau_1}} \end{bmatrix} \quad (3.1)$$

$$B \rightarrow B_d = \begin{bmatrix} \frac{\Delta t}{C_N} \\ -R_1(e^{-\frac{\Delta t}{\tau_1}} - 1) \end{bmatrix} \quad (3.2)$$

The resulting discrete state-space model has the following form:

$$\begin{bmatrix} SOC[k+1] \\ U_1[k+1] \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & e^{-\frac{\Delta t}{\tau_1}} \end{bmatrix} \begin{bmatrix} SOC[k] \\ U_1[k] \end{bmatrix} + \begin{bmatrix} \frac{\Delta t}{C_N} \\ -R_1(e^{-\frac{\Delta t}{\tau_1}} - 1) \end{bmatrix} I[k] \quad (3.3)$$

$$U_t[k] = \begin{bmatrix} \frac{f(SOC[k])}{SOC[k]} & 1 \end{bmatrix} \begin{bmatrix} SOC[k] \\ U_1[k] \end{bmatrix} + [R_0]I[k] \quad (3.4)$$

In Simulink the integrator was replaced with an equivalent Unit Delay block to be able to properly work in the time discrete domain (Figure 2.3). The solver was already configured correctly in the given template.

b)

The Kalman filter for SOC estimation is implemented as a discrete-time observer operating on the previously derived state-space battery model. As proposed in the exercise sheet, the filter is structured into three main stages: initialization, prediction, and update, followed by a sample-and-hold stage that provides the estimated state for the next iteration.

1. **Input and initialization stage:** In the input stage, the measured terminal

voltage U_t and the applied current I are forwarded to the Kalman filter. The initial SOC value is obtained from a *Unit Delay* block and is used to perform the first lookup of the SOC-dependent parameters R_0 , R_1 , τ_1 and U_{OCV} . These parameters define the initial system matrices used by the filter. The initial state is chosen as

$$\begin{bmatrix} SOC[0] \\ U_1[0] \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0 \end{bmatrix} \quad (3.5)$$

which reflects an approximate initial SOC assumption and a rested battery condition with zero RC overvoltage.

The process noise covariance matrix Q represents modeling uncertainties and unmodeled dynamics in the battery model. As recommended in the exercise, small diagonal values are used to indicate confidence in the state equations while still allowing the filter to adapt:

$$Q = \begin{bmatrix} 0.001 & 0 \\ 0 & 0.001 \end{bmatrix} \quad (3.6)$$

The observation noise covariance matrix R characterizes the measurement noise of the terminal voltage sensor. According to the task specification, it is initialized as

$$R = \begin{bmatrix} 0.001 \end{bmatrix} \quad (3.7)$$

The initial error covariance matrix P describes the uncertainty of the initial state estimate. Since the initial SOC is not known accurately and fast convergence is desired, relatively large diagonal values are chosen:

$$P = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \quad (3.8)$$

- 2. Prediction stage:** In the prediction stage, the state estimate and its associated uncertainty are propagated forward in time using the discrete-time battery model. Based on the current estimate $\hat{x}_k^+$, the predicted state $\hat{x}_{k+1}^-$ is computed as

$$\hat{x}_{k+1}^- = A\hat{x}_k^+ + B_k u_k \quad (3.9)$$

where the matrices A and B depend on the SOC-dependent parameters eval-

uated at the current estimate. This step represents the model-based evolution of the SOC and RC overvoltage under the applied current.

The corresponding predicted error covariance

$$P_{k+1}^- = AP_k^+ A^T + Q \quad (3.10)$$

accounts for the propagation of estimation uncertainty through the system dynamics and the addition of process noise.

3. **Measurement update stage:** In the update stage, the predicted state is corrected using the voltage measurement. The Kalman gain K_{k+1} is computed as

$$K_{k+1} = P_{k+1}^- C_{k+1}^T (R + C_{k+1} P_{k+1}^- C_{k+1}^T)^{-1} \quad (3.11)$$

The Kalman gain balances model confidence against measurement reliability and determines how strongly the measurement influences the state correction.

The updated state estimate is then obtained by

$$\hat{x}_{k+1}^+ = \hat{x}_{k+1}^- + K_{k+1} [y_{k+1} - (C_{k+1} \hat{x}_{k+1}^- + Du_{k+1})] \quad (3.12)$$

where the term in brackets represents the difference between the measured terminal voltage and the voltage predicted by the model. This correction step significantly improves the SOC estimate, especially in regions where the OCV–SOC relationship is steep.

Finally, the error covariance matrix is updated according to

$$P_{k+1}^+ = P_{k+1}^- - K_{k+1} C_{k+1} P_{k+1}^- \quad (3.13)$$

which reflects the reduced uncertainty after incorporating the measurement.

To initialize the Kalman filter, the script **InitSOCModel.m** (A.2) is executed prior to simulation. This script defines the matrices Q , R and P , ensuring consistent initialization of the filter parameters. The resulting Kalman filter provides a recursive estimate of the SOC and RC overvoltage based on the discrete-time state-space battery model. The Simulink design is shown in Figure 2.4.

c)

Figures 3.2, 3.3, and 3.1 show the estimated state of charge and internal voltages obtained using the Kalman filter, compared to the reference SOC computed by pure coulomb counting.

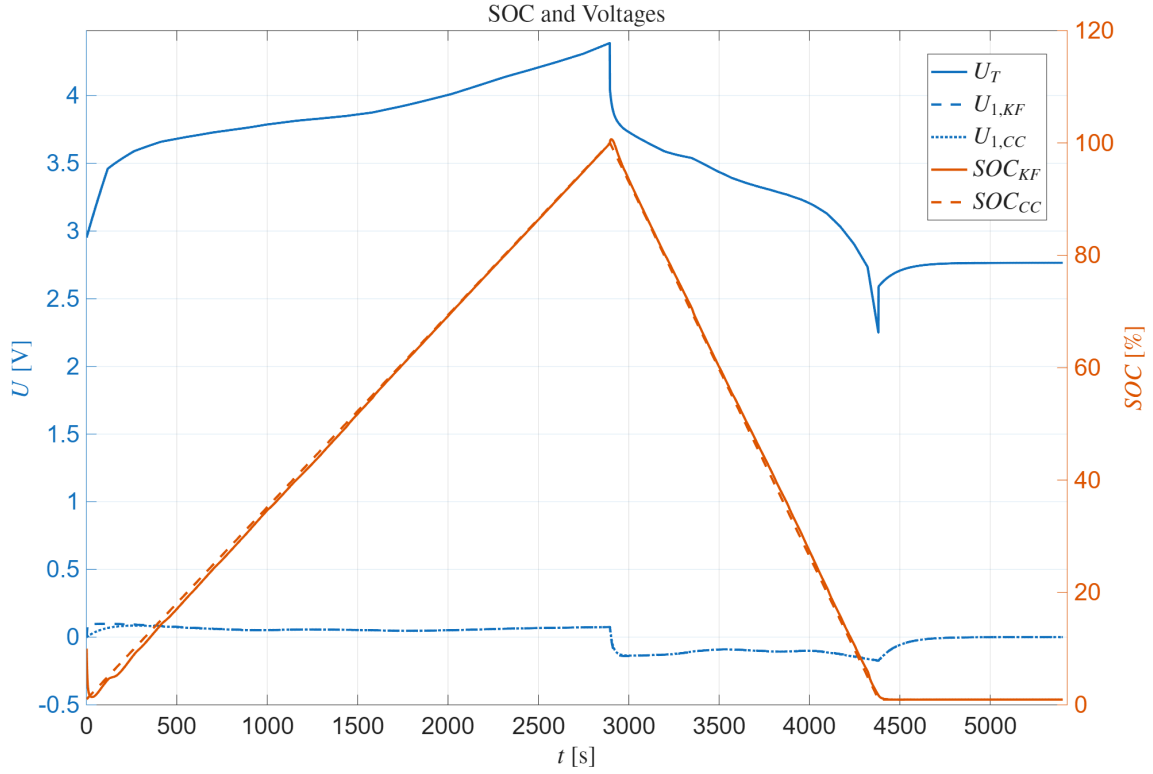


Figure 3.1.: Comparison of SOC and U_1 estimates (KF) and ground truth (CC)

At the beginning of the simulation, a noticeable deviation between the Kalman filter SOC estimate and the coulomb counting reference is observed (Figure 3.2). This deviation is expected and originates from the intentional mismatch between the true initial SOC and the initial state estimate used by the Kalman filter. While the reference SOC starts from the true value, the Kalman filter is initialized with an approximate SOC value and has to prove that it can correct the wrong initial guess.

During this initial phase, the Kalman filter rapidly corrects its estimate using the voltage measurement. The terminal voltage provides information about the SOC through the OCV–SOC relationship, which allows the filter to reduce the estimation error even when the initial SOC guess is inaccurate. As a result, the Kalman Filters SOC converges smoothly toward the ground truth SOC . This behavior illustrates the ability of the Kalman filter to compensate for initialization errors by combining model-based prediction with measurement feedback.

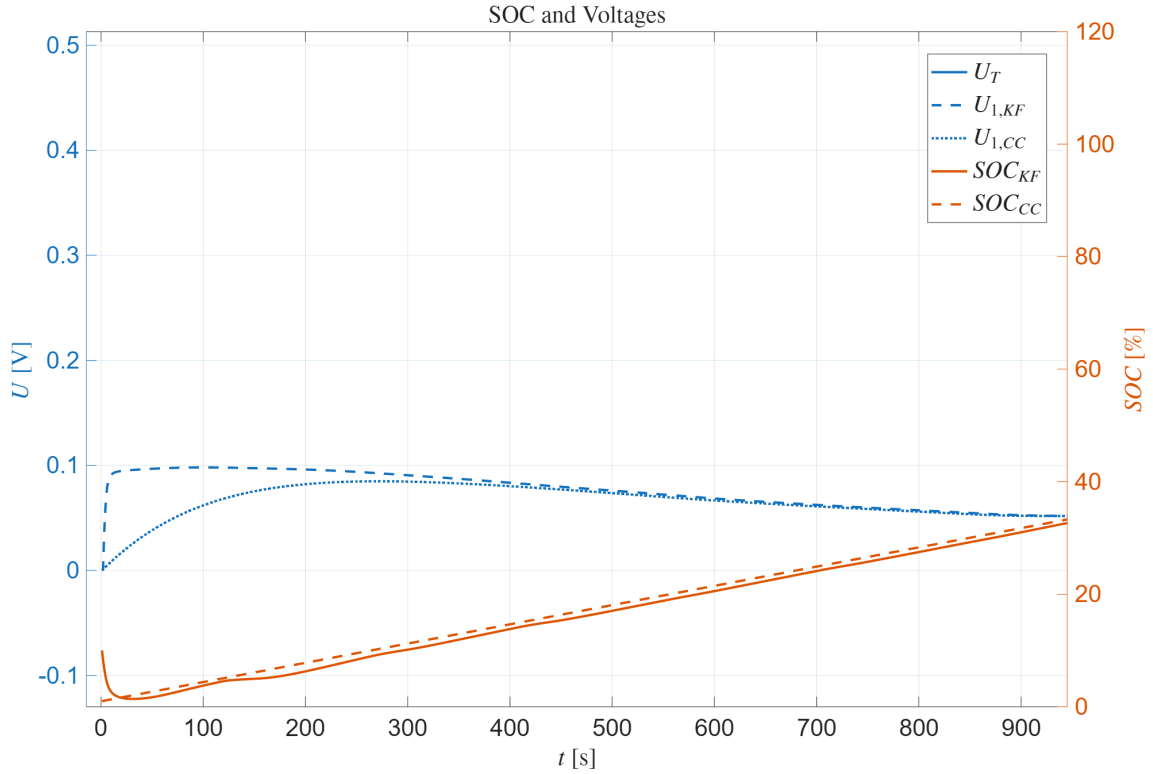


Figure 3.2.: Comparison of SOC and U_1 estimates (KF) and ground truth (CC) after initialization

A similar phase can be observed in the estimated RC voltage U_1 . Since the RC overvoltage state is also initialized with uncertainty, the filter requires a bit of time to align the internal state with the measured voltage dynamics. Once converged, the SOC and U_1 estimates closely follow the reference trajectories (CC).

During the charging phase, both SOC estimates increase linearly over time, as expected from the applied constant current. The close agreement during this phase confirms that the Kalman filter works as expected and does not introduce any errors or drift.

A pronounced overshoot in the Kalman filter SOC estimate is visible at the moment where the operating mode switches from charging to discharging. This effect is particularly visible in Figure 3.3. The overshoot does not occur in the coulomb counting reference and is therefore not a physical SOC change, but an estimation effect.

The overshoot is caused by the abrupt current reversal, which leads to a sudden change in terminal voltage. At the switching instant, the Kalman filter interprets the measured voltage deviation partly as a change in SOC , because the measurement equation links voltage to SOC via the OCV– SOC characteristic, leading to the observed overshoot.

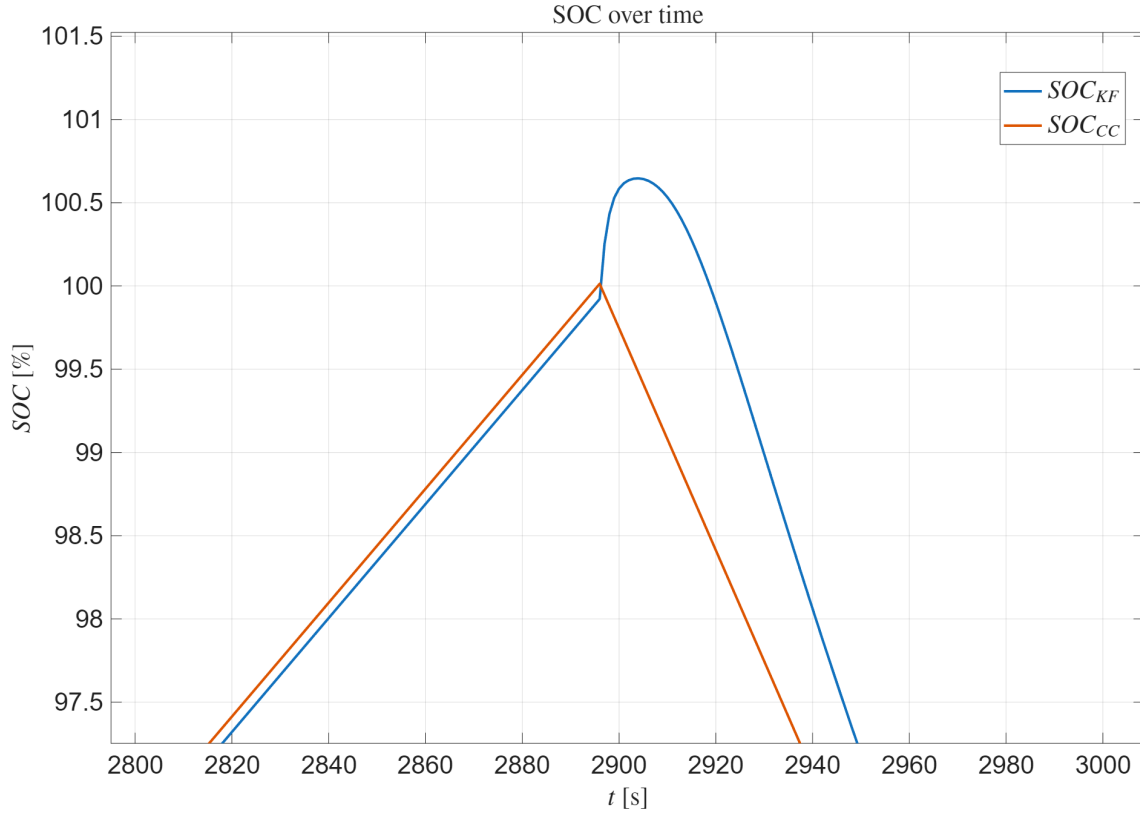


Figure 3.3.: *SOC* estimation overshoot at the charge to discharge switch

After the switch to discharging, the RC overvoltage decays and the voltage measurement becomes consistent with the new operating mode. Consequently, the Kalman filter corrects its *SOC* estimate and converges back towards the coulomb counting reference. Both *SOC* trajectories decrease linearly and remain closely aligned, indicating that the implemented Kalman Filter works as expected.

d)

We added two Uniform Random Number generators to add noise to our Simulink model. These number generators are limited to ± 0.02 V for U_t and ± 0.05 A for I and hence generate a white noise in the required value range. We add the noise to the input signals of our Kalman Filter. A manual switch is added to allow us to easily turn the noise on and off.

Compared to the noise-free case, Figure 3.4 shows that the noise from the inputs propagated into the *SOC* and U_1 estimates.

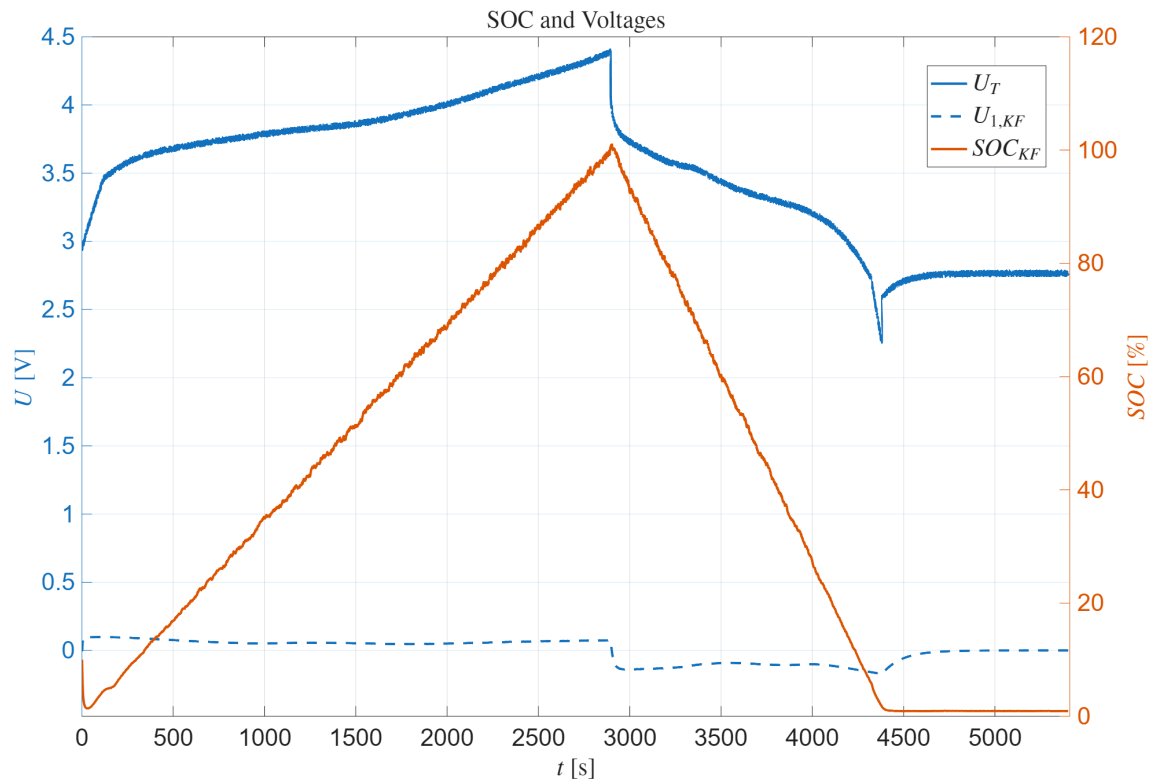


Figure 3.4.: SOC and U_1 estimates with input noise

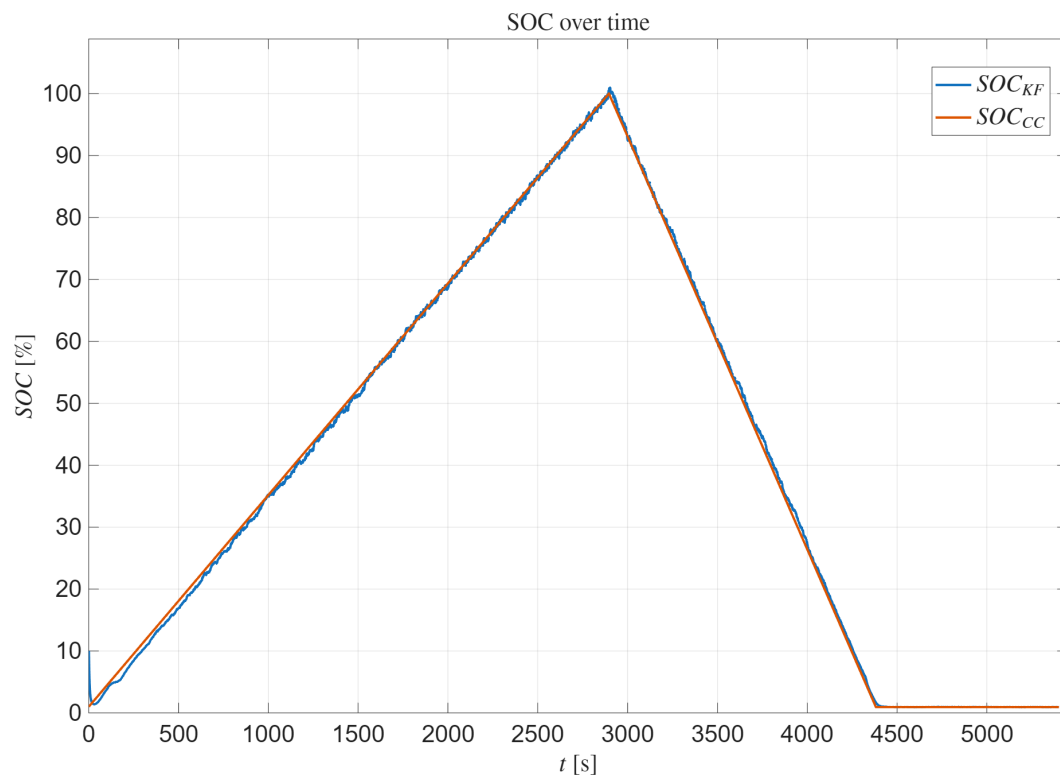


Figure 3.5.: SOC estimation and ground truth with input noise

Figures 3.5 and 3.4 show an increase in noise in the estimation with rising SOC levels. The effect is quite small but might be explained by the sensitivity of the terminal voltage to SOC , which is governed by the local slope of the OCV–SOC curve. Around regions where the OCV curve is steeper (in our case this is true for a higher SOC), small voltage changes lead to a larger change in SOC , so the Kalman update tends to transfer more of the measurement noise into SOC corrections.

Overall, the results show that the Kalman filter remains stable and continues to track the underlying SOC trajectory well even with noisy U_T and I . The noise-induced fluctuations are most visible in operating regions where the voltage measurement has a high sensitivity to the SOC and where the Kalman gain weights the measurement update strongly.

Bibliography

- [1] Product specification: Rechargeable Lithium Ion Battery 18650HE4 (2013)

List of Figures

1.1. BB1	2
2.1. Equivalent Circuit Model of the Cell	3
2.2. Voltages and SOC over time	8
2.3. Voltages and current over time	8
3.1. Comparison of SOC and U_1 estimates (KF) and ground truth (CC) .	13
3.2. Comparison of SOC and U_1 estimates (KF) and ground truth (CC) after initialization	14
3.3. SOC estimation overshoot at the charge to discharge switch	15
3.4. SOC and U_1 estimates with input noise	16
3.5. SOC estimation and ground truth with input noise	16
2.1. Simulink Parallel Model	25
2.2. Simulink Current Cycle Component	26
2.3. Simulink Discrete Parallel Model	27
2.4. Simulink Kalman Filter Model	28

A. Matlab-Code

Listing A.1: Initialization script InitParameterParallModel.m

```
%% Parameter initialization
% SOC and OCV refers to the SOC-OCV curve
% R0 is the internal resistance
% R1 and Tau1 describe the RC-circuit
% SOC_Param is the SOC vector for the parameters

par_model.Data = load('CellParameter.mat');

%% Equivalent parameters for the parallel connection
model
% CellParameter.mat contains single-cell values

% Number of parallel cells
par_model.np = 4;

% Resistances
par_model.R0 = par_model.Data.R0 ./ par_model.np; % [Ohm]
par_model.R1 = par_model.Data.R1 ./ par_model.np; % [Ohm]

% Time constant
par_model.Tau1 = par_model.Data.Tau1; % [s]

% Capacity
C_nom_single_cell_Ah = 2.5; % [Ah]
C_nom_single_cell_As = C_nom_single_cell_Ah * 3600; % [As]
par_model.CN = C_nom_single_cell_As * par_model.np; % [As]

% Efficiencies
par_model.eta_charge = 0.985;
```

```

par_model.eta_discharge = 1.0;
par_model.eta           = [par_model.eta_charge;
                           par_model.eta_discharge];

% OCV/SOC data (copied from single cell)
par_model.SOCData       = par_model.Data.SOC;
par_model.SOC_Param     = par_model.Data.SOC_Param;
par_model.OCVData       = par_model.Data.OCV;

%% Initialization of the state vector
SOC_init = 0.01;           % 1% SOC
U1_init  = 0.0;           % no initial
                        polarization voltage
par_model.x_init = [SOC_init; U1_init];

disp(par_model)

```

Listing A.2: Initialization script InitSOCModel.m

```

simulation.tsample = 1; % sampling time
kf.x_init = [0.1; 0];
kf.Q = [0.001 0; 0 0.001];
kf.R = [0.001];
kf.P_init = [0.5 0; 0 0.5];

```

Listing A.3: Function Block Scripts - Cell Model

```

% Function 1
function y = fcn(r1, tau, tsample)
y = -r1 * (exp(-tsample/tau) - 1);

% Function 2
function y = fcn(tau, tsample)
y = exp(-tsample/tau);

% Function 3
function y = fcn(I, charge, discharge)
if I > 0
    y = I * charge;

```



```
else
    y = I * discharge;
end
```

Listing A.4: Function Block Scripts - Kalman Filter

```
% Function 1
function P_minus= fcn(A, P_plus, Q)
A_T = transpose(A);
P_minus = A * P_plus * A_T + Q;

% Function 2
function K = fcn(P_minus, C, R)
C_T = transpose(C);
K = P_minus * C * inv(R + C_T * P_minus * C);

% Function 3
function x_hat_plus_next = fcn(x_hat_minus, K, U_t, C, D,
    I)
C_T = transpose(C);
x_hat_plus_next = x_hat_minus + K .* (U_t - (C_T *
    x_hat_minus + D * I));

% Function 4
function x_hat_minus = fcn(A, B, I, x_hat_plus)
x_hat_minus = A * x_hat_plus + B * I

% Function 5
function A = fcn(tau, tsample)
el_4 = exp(-tsample/tau);
A = [1 0; 0 el_4];

% Function 6
function B = fcn(r1, tau, cn1, tsample)
el_1 = tsample/cn1;
el_2 = -r1 * (exp(-tsample/tau) - 1);
B = [el_1; el_2];

% Function 7
```

```
function P_plus_next = fcn(P_minus, K, C)
C_T = transpose(C);
P_plus_next = P_minus - K .* C_T * P_minus;
```

B. Simulink Models

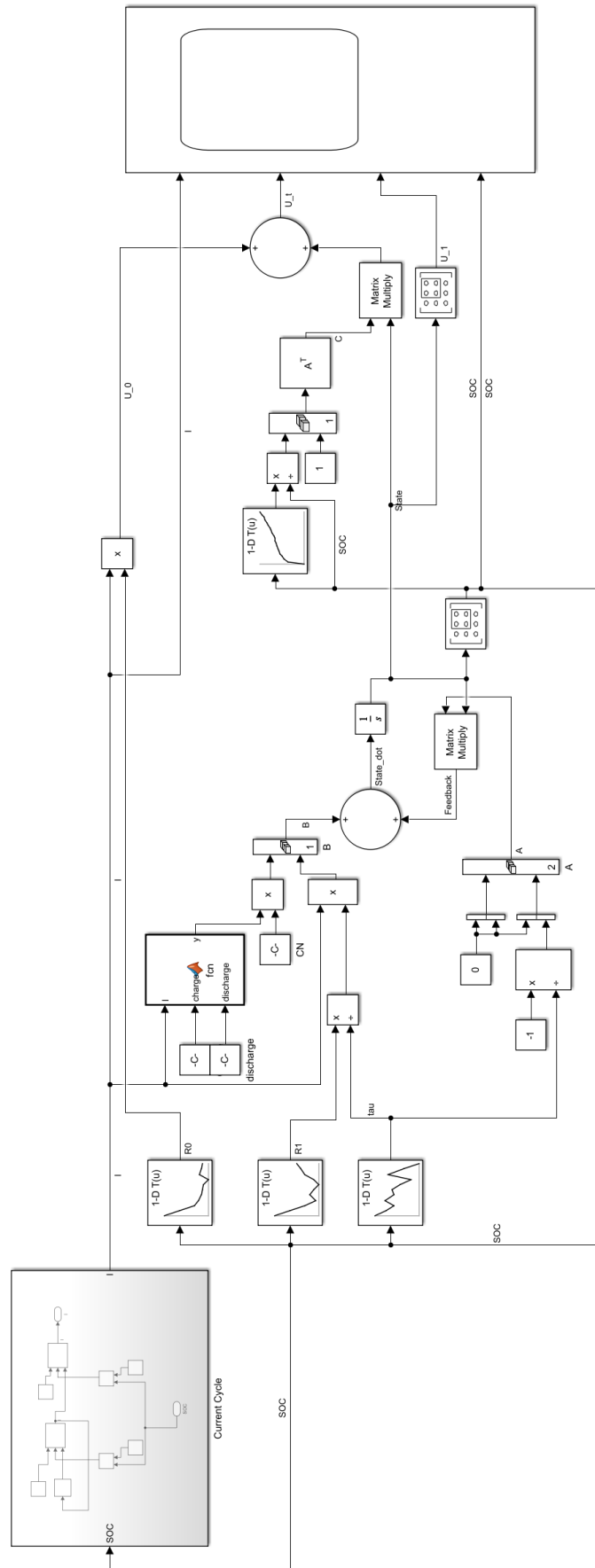


Figure 2.1.: Simulink Parallel Model

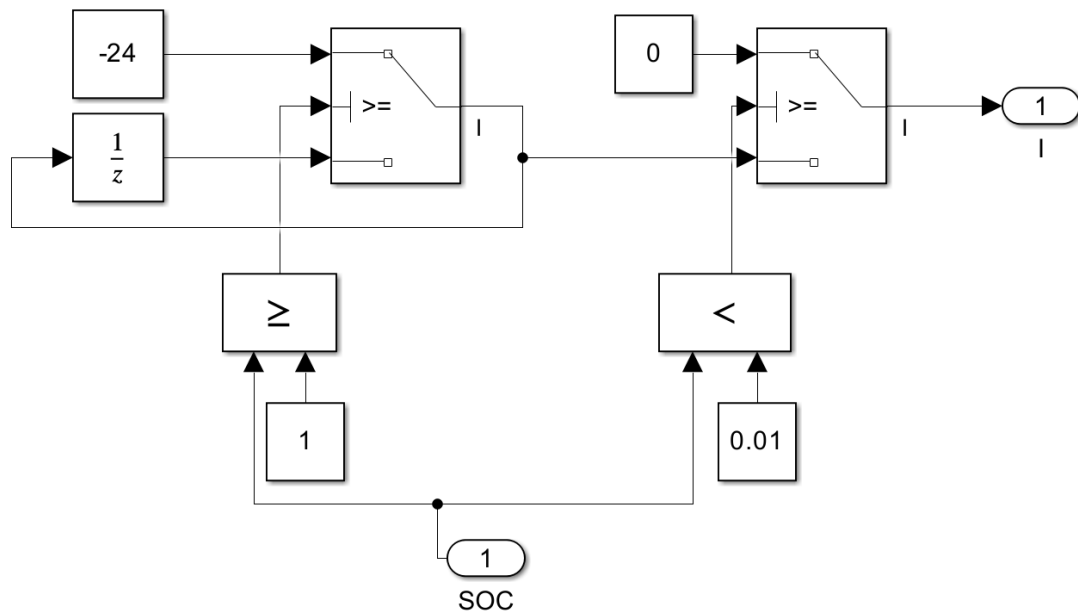


Figure 2.2.: Simulink Current Cycle Component

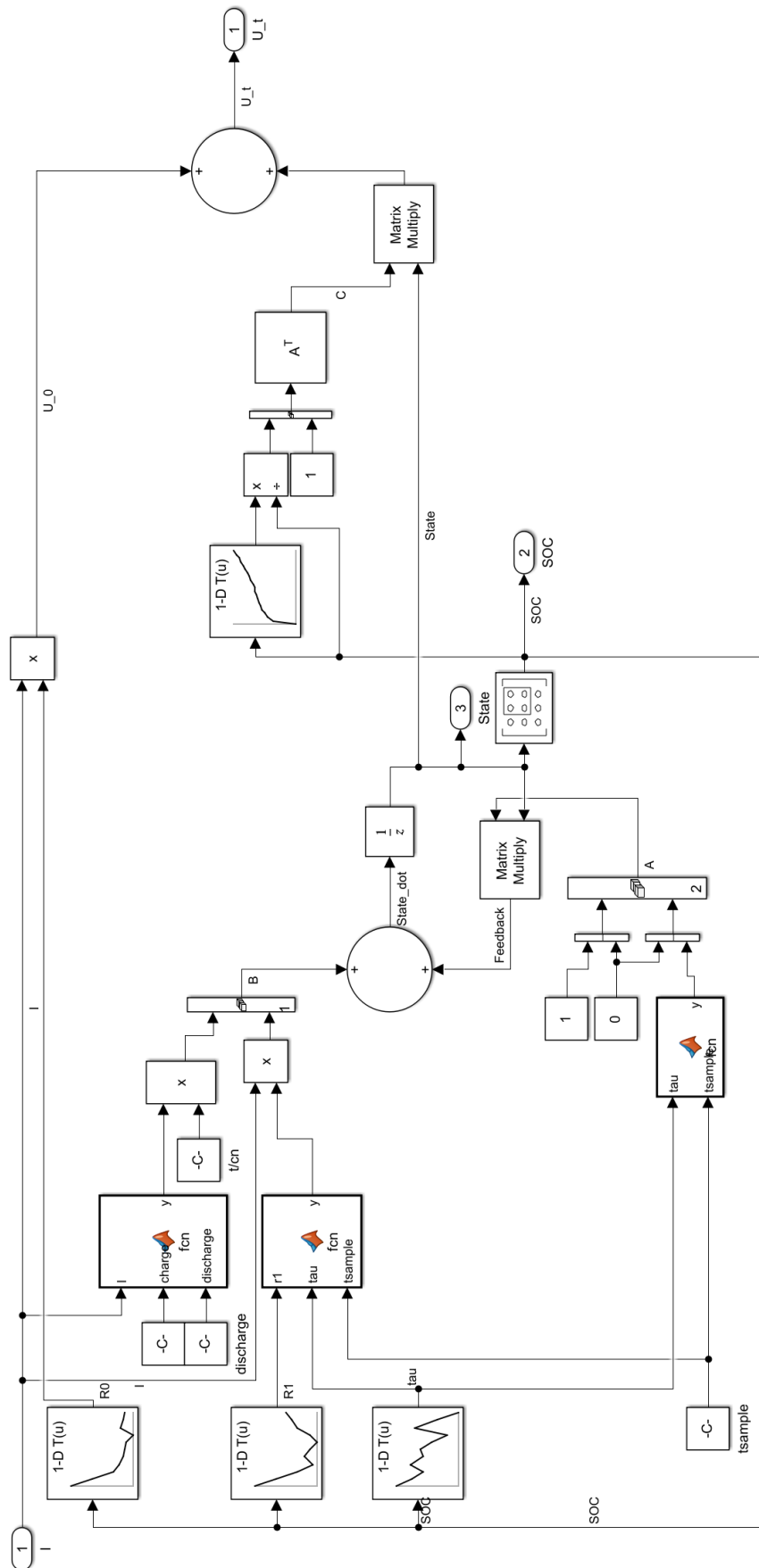


Figure 2.3.: Simulink Discrete Parallel Model

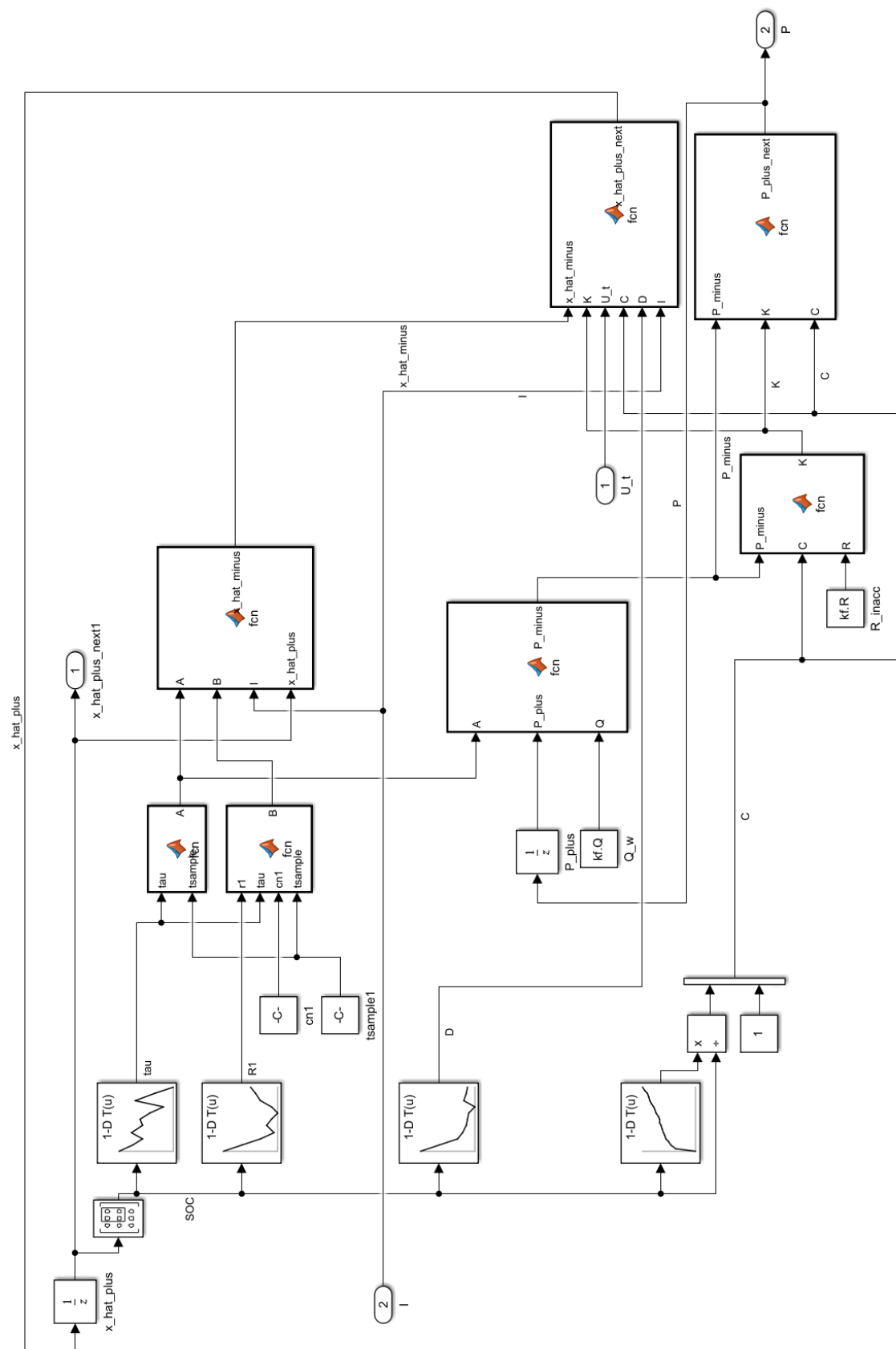


Figure 2.4.: Simulink Kalman Filter Model